cWB-LISA: A coherent WaveBurst-inspired pipeline for template-free searches in LISA data

Shubhanshu Tiwari
Department of Earth and Planetary Sciences, ETH Zurich.
(Dated: July 8, 2026)

The Laser Interferometer Space Antenna (LISA) will observe gravitational waves in the millihertz band using a moving triangular constellation and time-delay interferometry (TDI). This setting creates a qualitatively different environment from the ground-based detector networks for which coherent WaveBurst (cWB) was developed: the data are multi-channel TDI observables rather than geographically separated strain measurements, the detector response is strongly time dependent, and long-lived source confusion and instrumental effects are expected to be central analysis challenges. We describe the design of cWB-LISA, a modular pipeline intended to adapt the cWB philosophy of template-free, coherent time-frequency detection to LISA-like data. The pipeline is organized into four modules: simulation of synthetic data, TDI computation, coherence definition, and detection-statistic construction. The current implementation produces synthetic six-link data using waveform, orbit, response, and instrument simulators; computes pyTDI-based Michelson variables and orthogonal A/E/T channels; builds multi-resolution time-frequency maps; and prototypes a coronagraphic TDI path based on Sagnac variables and sky-dependent nulling. We emphasize the software and statistical interfaces required for a robust LISA burst search, identify the present mixed state of pyTDI and local static-TDI calculations, and define the development steps needed to turn the prototype into a validated coherent search.

I. INTRODUCTION

The coherent WaveBurst algorithm was designed for unmodeled gravitational-wave transients in ground-based detector networks. Its central idea is to search for excess power in a time-frequency representation and then rank coherent event candidates using consistency across multiple detectors, without requiring a phase-accurate waveform template for every source class [1]. This strategy is valuable whenever the signal morphology is uncertain, the source population is broad, or the goal is to preserve sensitivity to unexpected transients.

LISA requires a different realization of the same principle. The detector is a constellation of three spacecraft connected by six laser links, and the science data are formed by time-shifting and linearly combining inter-spacecraft measurements through TDI [2]. Common analysis bases include Michelson-like variables (X, Y, Z) and quasi-orthogonal combinations (A, E, T) [3]. In addition, Sagnac and coronagraphic TDI constructions can provide null or near-null information that may be useful for sky localization, glitch rejection, and model-agnostic consistency tests [4]. Therefore, a cWB-like search for LISA must be built around TDI observables and a LISA response model, not around independent strain channels from separated observatories.

This paper describes a proposed and partially implemented pipeline, here called cWB-LISA. The name is provisional but useful: it makes explicit both the inheritance from cWB and the adaptation to LISA-specific data products. The goal is not to claim a final detection statistic. Instead, the purpose is to define a clean architecture, document the current implementation status, and identify the technical decisions that must be stabilized before sensitivity or false-alarm claims can be made.

The paper is organized as follows. Section II states the design requirements. Section III defines the four target modules. Section IV describes the TDI layer and its current pyTDI status. Section V summarizes the time-frequency and clustering path. Section VI outlines the coherence problem and possible likelihood structure. Section VII gives the validation plan. Technical coding details are collected in the appendices.

II. DESIGN REQUIREMENTS

A LISA adaptation of cWB should satisfy four requirements.

First, the input data must be reproducible. The pipeline should be able to regenerate synthetic data from source parameters, waveform settings, orbit files, response settings, instrument-noise settings, and a random seed. This is essential because algorithmic changes in TDI, whitening, clustering, or ranking must be tested against fixed reference data.

Second, TDI computation must be treated as a first-class module rather than a hidden preprocessing detail. Laser-frequency noise cancellation, delay conventions, units, interpolation order, and generation choice enter before any time-frequency analysis is meaningful. The preferred strategy is to use pyTDI for standard combinations whenever possible, because pyTDI provides symbolic and numerical tools for LISA TDI combinations and ready-to-use standard observables [5].

Third, the definition of coherence must be LISA-specific. In ground-based cWB, coherence means consistency across detectors with different antenna patterns, noise spectra, and time delays. In LISA, the correspond-

ing objects are TDI channels whose noise covariance and response depend on TDI basis, frequency, sky direction, and the moving constellation. A coherent statistic must therefore be formulated in the TDI data space.

Fourth, detection statistics must be separated from implementation details. The ranking statistic, likelihood approximation, false-alarm calibration, and event summary should be defined independently from the scripts used to generate a given run. This separation is needed for reproducibility and for comparing alternative TDI bases, clustering methods, and background-estimation strategies.

III. TARGET PIPELINE ARCHITECTURE

The target architecture contains four modules, summarized in Table I. The modules are intentionally broad. Their purpose is to set stable interfaces before optimizing the individual algorithms.

A. Module 1: simulation of synthetic data

The first module generates the synthetic data used for development and validation. Conceptually, it contains a waveform simulator, a LISA orbit and response layer, and an instrument simulator. The intended output is the set of six directed inter-spacecraft link measurements, both with and without instrument noise. This six-link representation is the natural boundary between simulation and TDI.

For the current supermassive black-hole binary examples, the waveform stage can use a compact-binary waveform model such as IMRPhenomXPHM [6]. The response and instrument stages are handled through LISA simulation tools: LISA Orbits for spacecraft state vectors and delays, LISA GW Response for link responses, and LISA Instrument for the measurement chain and noise model [7–10]. The simulation module should not assume that all future sources are compact binaries; it should accept any source model that can be converted into link-level response products.

The minimum reproducibility contract is a manifest containing the source parameters, waveform model and version, orbit file, response settings, instrument settings, sample rate, time origin, science duration, burn-in duration, random seed, and output file names. This manifest should be written with every synthetic dataset.

B. Module 2: TDI computation from six-link data

The second module converts six-link data into TDI observables. It has two primary paths. The Michelson path maps the six links to pyTDI (X, Y, Z) and then to (A, E, T). The Sagnac path maps the six links to pyTDI (α, β, γ) and then to coronagraphic or other null combinations. These paths should share common configuration for units, interpolation order, delay derivatives, TDI generation, and output metadata.

The current production-level path is Michelson-to-A/E/T. The present code computes pyTDI Michelson observables for noisy instrument data and GW-only data, then forms A/E/T by a local orthonormal linear transform of X/Y/Z. The Sagnac path is less mature: pyTDI Sagnac variables are available in a diagnostic script, but the main coronagraphic prototype still uses a local static first-generation Sagnac construction for the kappa projection. This mixed state is acceptable for prototyping but not for final validation.

C. Module 3: coherence definition

The third module defines coherence. In a generic TDI basis, let $\mathbf{d}_p$ be the vector of whitened TDI coefficients in time-frequency pixel p, and let $\mathbf{R}_p(\Omega)$ be the response matrix for sky position and other extrinsic parameters denoted collectively by Ω. A simple coherent model can be written as

$$\mathbf{d}_p = \mathbf{R}_p(\Omega)\mathbf{h}_p + \mathbf{n}_p, \tag{1}$$

where $\mathbf{h}_p$ represents the two gravitational-wave polarizations or a reduced signal basis and $\mathbf{n}_p$ is noise with covariance $\mathbf{C}_p$.

For a fixed Ω, the coherent projection energy may be written schematically as

$$E_{\mathrm{coh}}(p,\Omega) = \mathbf{d}_p^\dagger \mathbf{C}_p^{-1}\mathbf{R}_p \left(\mathbf{R}_p^\dagger \mathbf{C}_p^{-1}\mathbf{R}_p\right)^{-1} \mathbf{R}_p^\dagger \mathbf{C}_p^{-1}\mathbf{d}_p. \tag{2}$$

The residual energy is the component orthogonal to the signal subspace,

$$E_{\mathrm{res}}(p,\Omega) = \mathbf{d}_p^\dagger \mathbf{C}_p^{-1}\mathbf{d}_p - E_{\mathrm{coh}}(p,\Omega). \tag{3}$$

Equations (2) and (3) are not yet the final statistic. They define a useful target interface: the coherence module should return quantities that separate signal-consistent energy from incoherent residuals, include null-channel information where available, and can be evaluated per pixel or per cluster.

D. Module 4: detection statistics and likelihood

The fourth module converts clusters and coherence features into event-level rankings. A minimal detection statistic could combine coherent energy, residual suppression, cluster size, time-frequency morphology, null-channel consistency, and sky-response concentration. A likelihood-based statistic would compare a signal-plus-noise model to a noise-only model over all pixels in a candidate cluster C,

$$\ln \Lambda_C = \sum_{p\in C} \left[\ln p(\mathbf{d}_p|\mathcal{H}_{\mathrm{sig}}) - \ln p(\mathbf{d}_p|\mathcal{H}_{\mathrm{noise}})\right]. \tag{4}$$

TABLE I. Target modules for the cWB-LISA pipeline. The coherence and likelihood modules are not yet final; they define the interface for the next development stage.

Module	Primary role	Main outputs
Simulation of synthetic data	Generate waveform, orbit, response, and instrument products from reproducible configuration files.	GW-only six-link data, noisy six-link data, orbit files, response metadata, run provenance.
TDI computation	Convert six-link products into standard and experimental TDI observables, using pyTDI as the source of truth where possible.	Michelson (X, Y, Z), orthogonal (A, E, T), Sagnac (α, β, γ), coronagraphic or null combinations, TDI diagnostics.
Coherence definition	Define signal consistency in the TDI data space.	Coherent energy, residual energy, null consistency, sky-dependent response consistency, per-pixel or per-cluster coherence features.
Detection statistics and signal likelihood	Rank candidate events and calibrate significance.	Event ranking statistic, likelihood or likelihood-ratio proxy, background-calibrated significance, injection-efficiency and sensitivity summaries.

The first implementation does not need to be a fully marginalized Bayesian likelihood. A robust ranking statistic, calibrated against noise-only and injection data, is a practical intermediate goal.

IV. TDI LAYER AND CURRENT IMPLEMENTATION STATUS

The most mature part of the current pipeline is the Michelson TDI path. The code uses pyTDI data interfaces for both noisy and GW-only products. Noisy TDI is computed from instrument HDF5 files, while GW-only TDI is computed from gravitational-wave response files and orbit files. The configured generation selects either first-generation or second-generation pyTDI Michelson observables. The resulting X/Y/Z streams are converted to A/E/T through

$$A = \frac{Z - X}{\sqrt{2}}, \tag{5}$$

$$E = \frac{X - 2Y + Z}{\sqrt{6}}, \tag{6}$$

$$T = \frac{X + Y + Z}{\sqrt{3}}. \tag{7}$$

For GW-only products, the current implementation multiplies fractional-frequency pyTDI outputs by the configured central laser frequency in order to store frequency-like units. This choice must remain explicit in output metadata.

The Sagnac and coronagraphic path is the major TDI development priority. Coronagraphic TDI introduces a sky-dependent combination, often denoted $\kappa(f, \beta, \lambda)$, with the useful property that the gravitational-wave contribution is suppressed when the tested sky direction approaches the true source direction [4]. In the prototype, candidate pixels are selected from pyTDI A/E time-frequency maps, while κ is evaluated using locally built static Sagnac variables from six-link data. A separate diagnostic script evaluates pyTDI Sagnac observables and compares them to the static construction. The next stable version should make pyTDI Sagnac products part of the same production TDI layer as Michelson/A/E/T.

V. TIME-FREQUENCY MAPS, CANDIDATE PIXELS, AND CLUSTERING

Following the cWB strategy, the pipeline searches in a time-frequency representation rather than in the raw time domain. The current prototype builds multi-resolution Wilson-Daubechies-Meyer-like maps for A and E channels and selects high-power pixels for downstream analysis. The exact transform implementation is currently a software dependency and should be treated as replaceable: the pipeline interface should accept WDM, wavelet, or other time-frequency maps as long as the resulting pixels carry channel, time, frequency, resolution, and normalization metadata.

For the coronagraphic prototype, the selected pixels are evaluated over a HEALPix sky grid. At each selected pixel and sky direction, the pipeline computes a kappa response or suppression measure. Each pixel is assigned features such as the best-null sky direction, the minimum kappa power, the median sky response, the sky contrast, and the sky sharpness. HDBSCAN clustering is then used to group pixels with similar sky-response features and time-frequency support [11, 12]. Multi-resolution runs repeat this procedure at several WDM resolutions and then merge coherent structures across resolutions.

This clustering path should be interpreted as a prototype. It is useful because it tests whether pixels from the same injected source prefer a common sky region under a model-agnostic nulling construction. It is not yet a final coherent likelihood, and it currently inherits the limitations of the static coronagraphic response model.

VI. COHERENCE AND DETECTION STATISTICS

The central open problem is to define a LISA-native coherent statistic. Three candidate ingredients are already clear.

First, the statistic should reward energy that is consistent with the response of a common gravitational-wave signal in a chosen TDI basis. This is the role of $E_{\rm coh}$ in Eq. (2).

Second, the statistic should penalize energy that cannot be explained by the same signal subspace. This is the role of $E_{\rm res}$ in Eq. (3) and of any explicit null-channel or coronagraphic veto.

Third, the statistic should use cluster-level structure. A real signal should not only be loud in isolated pixels; it should form a time-frequency morphology consistent across channels and, where applicable, across sky-dependent response features. Therefore, the event statistic should combine per-pixel coherence with cluster morphology and background calibration.

A practical first ranking statistic could take the form

$$\eta_C = f\left(\sum_{p\in C} E_{\rm coh}(p), \sum_{p\in C} E_{\rm res}(p), N_C, \Delta t_C, \Delta f_C, S_{\rm null}, S_{\rm sky}\right) \tag{8}$$

where N_C is the number of pixels in the cluster, Δt_C and Δf_C characterize its time-frequency extent, $S_{\rm null}$ measures null consistency, and $S_{\rm sky}$ measures sky-response concentration. The function f should initially be simple and interpretable. More flexible machine-learning or likelihood models should only be introduced after the physical diagnostics are stable.

VII. VALIDATION STRATEGY

The validation plan has five stages.

First, the pipeline must reproduce a fixed reference simulation from a clean configuration. This checks that waveform generation, response generation, instrument simulation, TDI computation, and output paths are deterministic under fixed seeds and versions.

Second, TDI products must be validated independently. The Michelson path should compare pyTDI X/Y/Z and A/E/T outputs across noisy and GW-only products, including unit checks and delay-derivative handling. The Sagnac path should compare pyTDI Sagnac observables to local static formulas only in regimes where the static assumptions are expected to hold.

Third, the time-frequency stage should be tested by injection recovery. A fixed injected signal should occupy stable regions of the A/E maps under changes in WDM resolution, whitening, crop, and normalization choices.

Fourth, the coherence and ranking stage should be tested against noise-only data and controlled injections. The relevant metrics are detection efficiency, false-alarm probability, cluster purity, sky-localization error for coronagraphic features, waveform-reconstruction error, and runtime.

Fifth, the background model must be made LISA-specific. Terrestrial detector time slides do not transfer directly to a single LISA constellation. Candidate alternatives include noise-only simulations, off-source regions, bootstrap resampling, phase randomization, and controlled source-subtraction studies.

VIII. DISCUSSION

The current prototype already contains the main ingredients of a cWB-inspired LISA search: synthetic link generation, pyTDI Michelson/A/E/T products, multi-resolution time-frequency maps, candidate-pixel selection, and exploratory coronagraphic clustering. The most important architectural decision is to make TDI computation a robust and explicit module. Without this, downstream coherence and likelihood definitions are not meaningful.

The largest current gap is the Sagnac/coronagraphic production path. The prototype mixes pyTDI A/E selection with local static Sagnac kappa computation. This is useful for rapid exploration, but a validated pipeline should either derive the coronagraphic response for the exact pyTDI Sagnac observables used in the data or clearly restrict the method to a static first-generation approximation with quantified errors.

A second gap is the lack of a final coherent statistic. The proposed data-model and projection equations define one possible starting point, but the final statistic must account for TDI noise covariance, sky-dependent LISA response, time-varying geometry, and source confusion. The correct development sequence is therefore: stabilize TDI, stabilize time-frequency maps, define an interpretable coherent statistic, and only then calibrate detection significance.

IX. CONCLUSION

cWB-LISA is a proposed modular pipeline for template-free gravitational-wave searches in LISA-like data. It preserves the cWB philosophy of coherent time-frequency event detection while replacing the ground-based detector network with a LISA-specific TDI data space. The pipeline is organized around four modules: synthetic-data simulation, TDI computation, coherence definition, and detection statistics. The current implementation has a robust pyTDI-based Michelson/A/E/T path and an exploratory coronagraphic clustering prototype. The next development priority is to make pyTDI Sagnac computation a validated production path and to define a LISA-native coherent statistic that can be calibrated on injections and noise backgrounds.

ACKNOWLEDGMENTS

This draft describes an in-development research pipeline. The implementation currently uses the LISA-TF repository structure and associated simulation, TDI, WDM, and coronagraphic prototype scripts. The author acknowledges the broader LISA simulation and cWB software ecosystems on which the design builds.

Appendix A: Current code map

The current repository is organized around reusable scripts and prototype analysis directories. The most relevant files are listed here because they define the present implementation state.

- `scripts/six_link_generation.py`: generates aligned waveform files, orbit files, six GW-only links, and six noisy links.
- `scripts/tdi_computation.py`: computes noisy and GW-only pyTDI Michelson X/Y/Z and derived A/E/T products.
- `scripts/wdm_computation.py`: computes multi-resolution time-frequency products and A/E maps.
- `scripts/lisa_link_simulation.py`: compatibility wrapper for the end-to-end link and TDI simulation path.
- `scripts/tdi_wdm_pipeline.py`: compatibility wrapper for TDI and WDM post-processing.
- `coronagraphic_tdi.py`: local static-geometry implementation of Sagnac, Michelson, A/E/T transforms, and coronagraphic kappa utilities.
- `choronagraphic-cluster/run_one_smbhb.py`: prototype pyTDI-A/E WDM selection and coronagraphic pixel clustering workflow. The directory name currently preserves the misspelling `choronagraphic-cluster`.
- `scripts/pytdi_sagnac_kappa_example.py`: diagnostic bridge comparing pyTDI Sagnac variables with the current static kappa projection.
- `test-run/generate_tdi_from_existing_h5.py`: post-processing utility to regenerate pyTDI XYZ/AET CSVs from existing HDF5 products.

Appendix B: pyTDI Michelson implementation details

The current Michelson production path imports `Data` from `pytdi` and imports `pytdi.michelson`. The generation selector maps the pipeline configuration to either `pytdi.michelson.X1/Y1/Z1` or `pytdi.michelson.X2/Y2/Z2`. Each combination is built with the data delays and sampling frequency. If available, delay derivatives are passed into the pyTDI build step.

Noisy TDI is computed from instrument output using `Data.from_instrument(..., signals="fluctuations", skipped=0)`. GW-only TDI is computed with `Data.from_gws(...)` using the gravitational-wave response file and orbit file. Both paths then crop to the science segment using metadata from the six-link CSV file. A simple FFT-domain frequency cut is applied after cropping.

The stored noisy output is

`outputs/tdi_michelson_XYZ_AET.csv`,

with columns for sample index, instrument sample index, waveform time, LISA time, X/Y/Z, and A/E/T. The stored GW-only output is

`outputs/tdi_gw_only_XYZ_AET.csv`,

with analogous columns and a `_gw_hz` suffix for GW-only TDI channels after conversion to frequency-like units.

The A/E/T transform is local but applied only after X/Y/Z are generated by pyTDI. Therefore, the current A/E/T products should be described as pyTDI-derived rather than independently implemented TDI combinations.

Appendix C: Sagnac and coronagraphic prototype details

The static coronagraphic module implements first-generation Sagnac combinations from six directed link streams. In frequency space, a Sagnac observable is represented schematically as

$$\alpha(f) = \sum_{\ell} s_\ell \exp[-2\pi i f \tau_\ell]\, y_\ell(f), \tag{C1}$$

with analogous expressions for β and γ. Here y_ℓ is a directed link measurement, s_ℓ is a sign, and τ_ℓ is the sum of static light-travel delays applied before that link contribution. The current implementation assumes an equilateral static LISA geometry for this construction.

The coronagraphic variable is computed as

$$\kappa(f,\Omega) = \mathbf{A}(f,\Omega) \cdot \begin{pmatrix} \alpha(f) \\ \beta(f) \\ \gamma(f) \end{pmatrix}, \tag{C2}$$

where $\mathbf{A}$ is built from the plus and cross response projectors for the tested sky direction $\Omega = (\beta, \lambda)$. In the idealized model, κ is suppressed when the tested direction approaches the source direction.

The pyTDI Sagnac diagnostic path evaluates `pytdi.sagnac.ALPHA*`, `BETA*`, and `GAMMA*` combinations for noisy or GW-only data. It can also request

`_ETA` combinations, but those require `eta_ij` measurements to be available in the input data. Current instrument products contain science, reference, and test-mass interferometer beatnotes rather than all required `eta_ij` measurements, so the `_ETA` path is not always available.

The next production version should choose one of two strategies. Either pyTDI Sagnac observables become the standard data vector and the coronagraphic response is derived for their exact convention, or the static local Sagnac implementation remains a clearly labeled approximation used only for controlled validation studies.

Appendix D: Time-frequency and clustering implementation details

The current WDM-selection path loads pyTDI-derived A/E products and crops them around the source or analysis segment. It optionally downsamples to a target cadence, computes WDM power maps for A and E, and forms an A+E selection map. Candidate pixels are chosen either by a fixed count or a top fraction of valid pixels, with optional frequency-band cuts and off-source normalization.

For each selected pixel, the coronagraphic prototype scans a HEALPix sky grid. The main saved products are:

- `candidate_pixels.csv`: selected time-frequency pixels;
- `healpix_sky_grid.csv`: tested sky directions;
- `kappa_features.csv`: best-null sky direction and response features for each pixel;
- `kappa_sky_responses.npz`: raw response matrix indexed by candidate pixel and sky direction;
- `pixel_clusters.csv`: HDBSCAN labels and per-pixel features;
- `summary.json`: run configuration and compact diagnostics.

Multi-resolution runs write the same products under one subdirectory per WDM resolution and then merge selected pixels across resolutions.

The current HDBSCAN feature vector includes sky-direction coordinates and logarithmic response features. This is a practical exploratory choice, not a final detection statistic. The final clustering interface should allow replacement by connected components, graph clustering, cWB-like constrained clustering, or a likelihood-driven event builder.

Appendix E: Immediate development to-do list

The following list is the recommended near-term development sequence.

1. Freeze one reference synthetic dataset and one reference configuration.
2. Treat pyTDI Michelson/A/E/T generation as the current validated baseline.
3. Promote pyTDI Sagnac generation into the same production TDI layer.
4. Validate Sagnac sign, scaling, Fourier, delay, unit, and generation conventions.
5. Decide whether local static Sagnac and Michelson formulas are retained only as reference tools.
6. Define standard output schemas for Michelson, A/E/T, Sagnac, and coronagraphic products.
7. Define the first LISA-native coherent statistic in a short mathematical note.
8. Implement a minimal event ranking statistic and compare it to incoherent A/E excess power.
9. Build noise-only and injection datasets for false-alarm and efficiency studies.
10. Produce a standard event report containing maps, clusters, sky-response diagnostics, ranking score, and metadata.

[1] M. Drago, V. Gayathri, S. Klimenko, C. Lazzaro, E. Milotti, G. Mitselmakher, V. Necula, B. O'Brian, G. A. Prodi, F. Salemi, M. Szczepanczyk, S. Tiwari, V. Tiwari, G. Vedovato, and I. Yakushin, Coherent waveburst, a pipeline for unmodeled gravitational-wave data analysis, SoftwareX **14**, 100678 (2021), arXiv:2006.12604 [gr-qc].

[2] M. Tinto and S. V. Dhurandhar, Time-delay interferometry, Living Reviews in Relativity **8**, 4 (2005), arXiv:gr-qc/0409034.

[3] P. Amaro-Seoane *et al.*, Laser interferometer space antenna (2017), arXiv:1702.00786 [astro-ph.IM].

[4] R. Costa Barroso, Y. Lemiere, F. Mauger, and Q. Baghi, Coronagraphic time-delay interferometry: characterization and updated geometric properties (2024), arXiv:2406.00190 [gr-qc].

[5] PyTDI developers, PyTDI: Manipulate and evaluate time-delay interferometry combinations, `https://pypi.org/project/pytdi/` (2026), accessed 2026-07-07.

[6] G. Pratten, C. Garcia-Quiros, M. Colleoni, A. Ramos-Buades, H. Estelles, M. Mateu-Lucena, R. Jaume, M. Haney, D. Keitel, J. E. Thompson, and S. Husa, Computationally efficient models for the dominant and sub-dominant harmonic modes of precessing binary black holes, Physical Review D **103**, 104056 (2021), arXiv:2004.06503 [gr-qc].

[7] LISA Simulation Expert Group, LISA Orbits, `https://pypi.org/project/lisaorbits/` (2026), accessed 2026-07-07.

[8] LISA Simulation Expert Group, LISA GW Response, `https://pypi.org/project/lisagwresponse/` (2026), accessed 2026-07-07.

[9] LISA Simulation Expert Group, LISA Instrument, `https://pypi.org/project/lisainstrument/` (2026), accessed 2026-07-07.

[10] J.-B. Bayle and O. Hartwig, Unified model for the lisa measurements and instrument simulations, Physical Review D **107**, 083019 (2023), arXiv:2212.05351 [gr-qc].

[11] R. J. G. B. Campello, D. Moulavi, and J. Sander, Density-based clustering based on hierarchical density estimates, in *Pacific-Asia Conference on Knowledge Discovery and Data Mining* (2013) pp. 160–172.

[12] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, Hierarchical density estimates for data clustering, visualization, and outlier detection, ACM Transactions on Knowledge Discovery from Data **10**, 5 (2015).